

ChordExec: A Decentralized Job Coordination Layer for Edge and IoT Environments Using Chord DHT

Neeraja Buch, Anupama Singh, Vi Thi Tuong Nguyen, Sreya Somisetty

Dept. of Computer Engineering

San Jose State University

San Jose, CA, USA

{author1, author2, author3, author4}@sjsu.edu

Abstract—Distributed job execution in edge and IoT environments demands high availability, horizontal scalability, and resilience to centralized coordination failures. Prevailing scheduler architectures rely on a master node for routing and dispatch, introducing a single point of failure ill-suited to dynamic, resource-constrained deployments. We present ChordExec, a fully decentralized job coordination layer in which the Chord Distributed Hash Table (DHT) serves as the control plane. Nodes self-organize into a consistent-hash ring, route task metadata in $O(\log N)$ hops via SHA-1-keyed finger tables, and replicate records across the successor chain using quorum-based durability semantics. An agent-driven orchestration layer augments pure Chord routing with load-aware placement by engineering job keys such that their SHA-1 digest maps to an under-loaded peer, preserving the Chord lookup invariant. The system exposes REST and gRPC interfaces for heterogeneous client integration. Evaluation on a multi-node ring demonstrates high throughput, complete job completion, and well-balanced cross-node distribution — a marked improvement over naive finger-table-only placement. Graceful node departure triggers ring repair and traffic redistribution within a single stabilization cycle. Remaining limitations in crash-consistency and ring scalability are identified as future work.

Index Terms—agent-based orchestration, decentralized coordination, distributed hash tables, edge computing, fault tolerance,

Stoica et al., arranges nodes in a consistent-hash ring and resolves key ownership in $O(\log N)$ hops using a finger table of size M . Chord has been widely studied as a lookup mechanism, yet its application as a job execution control plane — routing, storing, and replicating task metadata without any central participant — remains underexplored in the systems literature, particularly for real workloads with heterogeneous load and dynamic membership.

This paper describes ChordExec, a distributed job execution platform that eliminates the central coordinator entirely by making the Chord ring itself the scheduling substrate. Our design contributions are as follows:

- **Chord as a control plane.** Every task record is stored under the key `task:{task_id}`, hashed with SHA-1 and routed to its responsible node via `find_successor`. Any node in the ring can accept a client request; no distinguished master is required.
- **Successor-chain replication with quorum.** After a primary write, ChordExec replicates to the $k - 1$ nearest successors and labels the write `COMPLETE` when $\lceil k/2 \rceil$ replicas confirm, or `DEGRADED` otherwise. Reads first try the primary, then fall back to replicas — providing availability in the presence of partial failures.
- **Agent-guided load-aware placement.** A periodic `AgentLoop` performs a full successor-chain walk to collect queue-depth metrics from all reachable nodes. When submitting a job, an orchestration agent selects the least-loaded peer and engineers the job's key — iterating candidate UUIDs until $\text{sha1_id}(\text{"job:{uuid}"}) \bmod 2^M \equiv \text{target_node_id} \bmod 2^M$ — so the Chord lookup routes the job to the intended node without bypassing ring invariants.

Dual REST and gRPC API. A Flask-based HTTP layer handles Chord RPCs, client-facing task and job endpoints, and an administrative interface for ring introspection and node departure. A gRPC service mirrors the task API for latency-sensitive clients.

We evaluate ChordExec on a seven-node ring running on a single host (loopback networking), measuring throughput, job-

I. INTRODUCTION

Modern compute infrastructure increasingly extends beyond the data center into edge networks, IoT gateways, and mobile cloudlets. In these environments, workloads must be orchestrated across dozens to hundreds of loosely coupled nodes that join and leave unpredictably. Classical job schedulers — Apache YARN [1], Kubernetes [2], Mesos [3] — address this by designating a central resource manager or control plane. While effective at data-center scale with stable membership, this architecture creates two structural liabilities in edge settings: (i) the coordinator is a single point of failure whose loss stalls the entire system, and (ii) all scheduling decisions funnel through one bottleneck, limiting horizontal scalability.

Peer-to-peer Distributed Hash Tables offer an alternative coordination substrate. The Chord protocol [4], proposed by

completion rate, distribution balance across replica configurations (1×, 2×, 3×), and behavior under graceful node removal. Our key findings are: (i) a 100% job-completion rate at 13 jobs/sec execution throughput, (ii) a distribution variance of 8.3% after deploying the full ring walk — compared to 62.5% when metrics were naively bounded to the local finger table — and (iii) successful ring repair and 70% traffic redirection after a graceful node departure.

The remainder of this paper is organized as follows. Section II surveys existing decentralized and DHT-based coordination approaches. Section III details the system design of ChordExec. Section IV presents our experimental methodology and results. Section V discusses limitations and directions for future work, and Section VI concludes.

II. RELATED WORK

We survey prior work across three axes: centralized job scheduling platforms, peer-to-peer DHT protocols, and DHT-based storage and task-coordination systems.

A. Centralized Job Scheduling Platforms

The dominant paradigm in production distributed computing is a two-tier architecture: a centralized resource manager and a set of worker nodes. **Apache Hadoop YARN** [1] separates resource management from job lifecycle management through a ResourceManager/NodeManager hierarchy. All resource-allocation decisions flow through the ResourceManager, which is both a performance bottleneck and a single point of failure; although YARN supports high availability via ZooKeeper-based failover [5], recovery involves downtime and requires external consensus infrastructure. **Apache Mesos** [3] takes a similar approach through a master that issues resource offers to registered frameworks. **Kubernetes** [2] centralizes scheduling in the *kube-scheduler* and maintains cluster state in an etcd [6] cluster; the control plane is replicated but fundamentally centralized, as all scheduling decisions require quorum writes to etcd.

HashiCorp Nomad [4] uses a Raft-based consensus group for scheduling decisions, which reduces single-node risk but still requires a designated set of coordinator nodes to be reachable.

In contrast, ChordExec places no coordinator requirement on the system. Every node is structurally identical; any node can accept and route a task request, and ring membership changes are handled by the Chord maintenance protocol itself rather than by an external consensus layer.

B. Peer-to-Peer DHT Protocols

Distributed Hash Tables were designed to provide scalable, decentralized key-value lookup without any central directory. The four canonical structured overlay protocols are Chord [7], Kademlia [8], Pastry [9], and CAN [10].

Chord [7] assigns each node and key an m -bit identifier via SHA-1 and organizes nodes in a clockwise ring. A finger table of m entries allows any node to route a lookup to the key’s successor in $O(\log N)$ hops. Ring maintenance—*stabilize*,

fix_fingers, *check_predecessor*—enables the ring to self-heal as nodes join and leave. Our system implements the Chord protocol directly as its routing core.

Kademlia [8] uses XOR as a distance metric and routes through a binary tree of k -buckets. Kademlia is widely deployed (BitTorrent DHT, IPFS) and provides parallelized lookups, but its XOR metric lacks the clean ring structure that makes Chord’s successor chain a natural replication target. ChordExec exploits the linearly ordered successor list to implement an ordered replication chain without additional metadata.

Pastry [9] routes based on shared prefix length and supports a *leaf set*—the k nearest nodes by identifier—which is used for replication in the Past storage system [15]. Pastry’s leaf set is structurally similar to our successor chain, but Pastry’s routing tables are more complex and its original design targets large-scale Internet overlays rather than infrastructure-layer job execution.

CAN [10] partitions a d -dimensional Cartesian coordinate space; nodes own zones rather than ring segments. CAN provides $O(d \cdot N^{1/d})$ routing but lacks the natural ring ordering that simplifies successor-chain replication.

None of these protocols embed job execution semantics—task schema validation, job lifecycle state machines, worker thread pools, or placement heuristics—in the overlay layer. ChordExec builds these concerns on top of Chord while keeping the routing layer unchanged.

C. DHT-Based Storage and Coordination Systems

Several production systems use DHT-inspired consistent hashing for data placement without running a full DHT routing protocol.

Amazon DynamoDB [12] uses consistent hashing to partition data across nodes and replicates each key to N successive nodes on a virtual-node ring. Reads and writes use a sloppy quorum ($W + R > N$) with hinted handoff for fault tolerance. DynamoDB’s replication model is the closest prior art to our successor-chain quorum: both walk the ordered ring to find replica targets and label writes as degraded when the replica count falls below threshold. DynamoDB, however, operates with a proprietary coordinator layer and virtual nodes per physical host—abstractions absent in our prototype.

Apache Cassandra [13] also uses a token ring with configurable replication factor and tunable consistency levels. Like DynamoDB, Cassandra is a storage system and does not provide a job execution abstraction.

Chord-based file systems such as **CFS** [14] use Chord for block location and DHash for replication, demonstrating that successor-chain replication is feasible and correct under churn. ChordExec applies the same principle to task metadata rather than file blocks.

D. Edge and IoT Task Scheduling

The emergence of fog and edge computing has motivated scheduling research that accounts for device heterogeneity, unstable connectivity, and geographically distributed workloads.

TABLE I
COMPARISON OF COORDINATION APPROACHES

System	No coord.	DHT routing	Job exec.	Succ. repl.	Load-aware
YARN / Mesos / K8s	×	×	✓	×	✓
DynamoDB / Cassandra	~	~	×	✓	×
Chord / Kademlia	✓	✓	×	×	×
CFS / PAST	✓	✓	×	✓	×
FogBus / iFogSim	×	×	✓	×	~
ChordExec (ours)	✓	✓	✓	✓	✓

~ Partial support. YARN/Mesos/K8s load-awareness is centralized.

FogBus [17] and **iFogSim** [18] provide simulation frameworks for edge-cloud task offloading, but both assume a hierarchical control structure with cloud-tier coordinators. **ENORM** [19] proposes edge node resource management through negotiation but relies on a broker. P2P-based edge orchestration work, such as that of Confais *et al.* [20], applies Kademlia-style DHT routing to object storage at the edge, but does not address job execution lifecycle, replication quorum semantics, or agent-guided load-aware placement.

E. Summary and Positioning

Table I summarizes the positioning of ChordExec relative to representative prior systems along the axes most relevant to decentralized edge job execution.

To our knowledge, no prior system combines: (i) Chord DHT routing as the exclusive control plane for job metadata; (ii) successor-chain replication with quorum-labeled durability; (iii) agent-loop metrics collection via full ring walk; and (iv) key engineering for load-aware placement that preserves the Chord lookup invariant. ChordExec addresses this gap.

III. SYSTEM DESIGN

Fig. 1 shows the layered architecture of ChordExec. From bottom to top: a *transport layer* carries HTTP RPCs between peers; a *Chord ring layer* provides decentralized routing and ring maintenance; a *storage layer* routes, validates, and replicates task metadata; a *job execution layer* claims and executes work from the local store; and an *agent orchestration layer* observes the full ring and steers placement decisions. A *dual API layer* (REST + gRPC) accepts requests from any external client.

A. Chord Ring Layer

1) *Node Identifiers and Key Space*: Every node n is assigned an m -bit identifier $id(n) = \text{SHA-1}(\text{address}(n)) \bmod 2^m$. Our implementation uses $m = 8$ (a 256-position ring), which is sufficient for single-host testing while preserving the full Chord routing semantics; a production deployment would use $m = 160$ to match SHA-1's full output width. Keys are hashed identically: $id(k) = \text{SHA-1}(k) \bmod 2^m$.

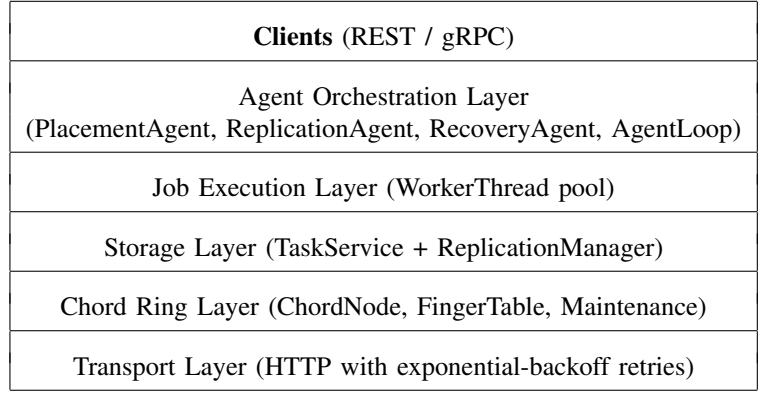


Fig. 1. ChordExec layered architecture.

Algorithm 1: find_successor(n, k)

Input: Node n , target key k
Output: Successor node reference ($id, address$)

```

1 if  $k \in (id(n), id(succ(n)))$  (wrap-aware) then
2   return  $succ(n)$ 
3 end
4  $n' \leftarrow \text{closest\_preceding\_node}(n, k)$ 
5 if  $id(n') = id(n)$  then
6   return  $succ(n)$  // single-node ring
7 end
8 return RPC  $n'.\text{find\_successor}(k)$ 

```

Algorithm 2: closest_preceding_node(n, k)

Input: Node n , target key k
Output: Best preceding node for k

```

1 for  $i \leftarrow m - 1$  to 0 do
2   if  $finger[i].id \in (id(n), k)$  (wrap-aware) then
3     return  $finger[i]$ 
4   end
5 end
6 return  $n$ 

```

2) *Finger Table*: Each node n maintains a finger table of m entries. Entry i stores:

$$finger[i].start = (n + 2^i) \bmod 2^m, \quad 0 \leq i < m$$

and a pointer ($id, address$) to the first node on the ring that is at or clockwise of $finger[i].start$. Entry 0 is the immediate successor $succ(n)$.

3) *Lookup: find_successor*: Algorithm 1 gives the lookup procedure. If the target key k falls in the half-open arc $(id(n), id(succ(n)))$, the successor is returned immediately. Otherwise, the node with the largest finger ID that strictly precedes k on the ring ($\text{closest_preceding_node}$) is found and a remote find_successor RPC is issued. On RPC failure the local successor is returned as a safe fallback to prevent infinite recursion in a degraded ring.

B. Ring Maintenance

Three background daemons run concurrently on every node to keep the ring consistent under membership changes.

Algorithm 3: stabilize(n)

```

Input: Node  $n$ 
1  $x \leftarrow \text{RPC } \text{succ}(n).\text{get\_predecessor}()$ 
2 if  $x \neq \emptyset$  and  $\text{id}(x) \in (\text{id}(n), \text{id}(\text{succ}(n)))$  then
3    $\text{succ}(n) \leftarrow x$ 
4 end
5  $\text{RPC } \text{succ}(n).\text{notify}(n)$ 
  // Successor unreachable: scan fingers for fallback
6 for  $i \leftarrow m-1$  to 1 do
7   if  $\text{ping}(\text{finger}[i].\text{address})$  succeeds then
8      $\text{succ}(n) \leftarrow \text{finger}[i]$ 
9     break
10  end
11 end

```

1) *stabilize*: Algorithm 3 corrects the successor pointer after a new node inserts itself between n and $\text{succ}(n)$. The node asks its current successor for its predecessor x ; if x lies strictly between n and $\text{succ}(n)$, the successor is updated. On successor failure the node scans its finger table from the highest index downward to find the first reachable node as a fallback successor, preventing the ring from partitioning around a single dead node.

2) *fix_fingers*: Each invocation selects one finger index i uniformly at random from $[1, m-1]$ and refreshes it by calling $\text{finger}[i] \leftarrow \text{find_successor}(\text{finger}[i].\text{start})$. Over time all m fingers converge to correct values.

3) *check_predecessor*: The node pings its predecessor; if the ping fails, the predecessor pointer is cleared to $\emptyset$ so that the next node to call `notify` can establish itself as the new predecessor.

4) *Join and Graceful Leave*: When node n joins via a known bootstrap address b : (i) $\text{succ}(n) \leftarrow \text{RPC } b.\text{find_successor}(\text{id}(n))$; (ii) stabilization propagates n 's presence to its new predecessor.

On graceful departure, n : (i) bulk-transfers its local data store to $\text{succ}(n)$; (ii) updates $\text{succ}(n)$'s predecessor pointer to $\text{pred}(n)$; and (iii) updates $\text{pred}(n)$'s successor pointer to $\text{succ}(n)$. This three-step handoff ensures no data is lost and the ring re-closes correctly.

C. Task Routing and Storage

The *TaskService* layer adds typed, validated task records on top of the raw Chord key-value store. Every task is stored under the canonical key `task:{task_id}`. The routing and write path is:

- 1) **Key derivation.** $\text{key_id} = \text{SHA-1}(\text{task}:\{\text{task_id}\}) \bmod 2^m$.
- 2) **Owner resolution.** $\text{primary} \leftarrow \text{find_successor}(\text{key_id})$, issued on the entry-point node. Any node may serve as entry point.
- 3) **Primary write.** The task record is sent to the primary. If the primary is unreachable (due to concurrent ring stabilization), the system re-resolves the owner and retries up to $R_p = 2$ additional times with exponential back-off ($\Delta_0 = 0.2$ s, doubling each attempt).

Algorithm 4: Successor-Chain Replication Write

```

Input: Task key  $K$ , record  $R$ , primary  $P$ , factor  $k$ 
Output: Replication result with quorum label
1  $\text{chain} \leftarrow \text{get\_successor\_chain}(P, k)$ 
2  $\text{written} \leftarrow 1$  // primary already persisted
3 foreach replica  $r \in \text{chain}[1:]$  do
4    $\text{ok} \leftarrow \text{FALSE}$ 
5   for attempt  $\leftarrow 0$  to  $R_r$  do
6     if  $\text{RPC } r.\text{put\_replica}(K, R)$  succeeds then
7        $\text{written} += 1$ 
8        $\text{ok} \leftarrow \text{TRUE}$ 
9       break
10    end
11     $\text{sleep}(\delta_0 \cdot 2^{\text{attempt}})$ 
12  end
13  if not  $\text{ok}$  then
14    log permanent failure for  $r$ 
15  end
16 end
17  $q \leftarrow \lceil k/2 \rceil$ 
18 return  $\text{written} \geq q$  ? COMPLETE : DEGRADED

```

4) **Replication** (Section III-D).

5) **Read with replica fallback.** A read first contacts the primary; on any exception or miss, it falls back to the replica chain, ensuring availability when the primary has crashed between write and read.

All task records carry a schema-validated JSON structure including `task_id`, `status`, `owner_node`, `replica_nodes`, `replication_factor`, and optional placement fields. Soft deletes set `status = DELETED` and propagate the tombstone to replicas; hard deletes physically remove the key from primary and all replicas.

D. Successor-Chain Replication with Quorum

After a successful primary write the *ReplicationManager* constructs a successor chain of length k by following live `/chord/state` successor pointers from the primary outward (Algorithm 4).

Each replica write is retried up to $R_r = 2$ additional times with exponential back-off ($\delta_0 = 0.2$ s). This is distinct from the transport-level retries, which handle transient connection errors; the replication-level retries handle application-level transient errors such as HTTP 503 responses or remote lock contention.

The quorum threshold is $q = \lceil k/2 \rceil$. If the total number of successful writes (primary + replicas) reaches q , the record is labeled COMPLETE; otherwise DEGRADED. For example, with $k = 3$ a single replica failure still yields COMPLETE ($2 \geq 2$), while two replica failures yield DEGRADED ($1 < 2$).

E. Agent Orchestration Layer

ChordExec includes three specialized agents and a monitoring loop, all sharing a common decision log (`agent_decisions.jsonl`) for auditability. Agents operate in two modes controlled by the `AGENT_STRATEGY` environment variable: `llm` (uses the Claude API via tool-forced calls with a cached system prompt) or `heuristic`

(pure local logic, no external dependency). If the LLM call fails for any reason, the agent transparently falls back to the heuristic.

1) *PlacementAgent*: Given a job record and a list of ring-node metrics snapshots $\{(id_i, queue_depth_i, jobs_failed_i)\}$, the *PlacementAgent* selects the target node n^* that minimizes queue depth (and secondarily, failure count):

$$n^* = \arg \min_i (queue_depth_i, jobs_failed_i).$$

2) *ReplicationAgent*: The *ReplicationAgent* receives the job record, node metrics, and a caller-specified minimum replication factor. It may raise the factor for high-priority job types (e.g., `compute`) when sufficient nodes are available, and selects the replica set from the sorted node list.

3) *RecoveryAgent*: When a node failure is detected, the *RecoveryAgent* receives the list of orphaned jobs and surviving nodes. Rather than routing all orphans to the Chord successor (which would concentrate load), it distributes orphaned jobs round-robin across survivors sorted by queue depth, achieving balanced redistribution without any global coordinator.

4) *AgentLoop*: The *AgentLoop* runs as a daemon thread on every node with a configurable polling interval (default 5 s). Each tick it performs a full ring walk—following live successor pointers via HTTP GET `/chord/state` starting from its own successor—and fetches GET `/metrics` from every reachable node. When a node in the walk is unreachable, the loop does not abort; instead, it attempts to skip forward using a finger-table entry that lies beyond the dead node. This design is critical: the original implementation that followed only finger-table peers (at most $\log_2 N$ nodes) caused 62.5% of jobs to concentrate on a single node. The corrected full-ring walk restored visibility to all 7 nodes and reduced distribution variance from 62.5% to 8.3% (Section ??).

F. Key Engineering for Load-Aware Placement

A fundamental tension in DHT-based job execution is that Chord routing is deterministic: $\text{SHA-1}(\text{job}:\{\text{uuid}\})$ determines which node receives a job, with no regard for that node's current load. Pure Chord routing is therefore incompatible with load-aware placement.

ChordExec resolves this tension through *key engineering* (Algorithm 5). After the *PlacementAgent* selects a target node n^* , the system iterates random UUIDs until it finds one whose SHA-1 hash lands on n^* in the 2^m -position ring. The resulting key is used for both storage and lookup, so all subsequent `find_successor` calls still route to n^* through the normal Chord mechanism. No bypass, no shadow table, no modification to the routing protocol.

With $m = 8$ (ring size 256), the expected number of UUID trials before a match is 256. In practice, the default limit of 512 attempts succeeds with probability $1 - (255/256)^{512} \approx 0.865$. For production rings with $m = 160$, this brute-force approach is not feasible; a cryptographic key derivation function (KDF) with a ring-position constraint would be required.

Algorithm 5: `make_job_key_for( $n^*$ , $ring\_size$ )`

Input: Target node n^* , ring size $2^m = ring_size$, max attempts T

Output: Job key string whose SHA-1 routes to n^*

```

1 for attempt ← 1 to T do
2   u ← uuid4()
3   key ← "job:" + u
4   if SHA-1(key) mod ring_size ≡ n* mod ring_size
5     then
6       return key
7   end
8 return "job:" + uuid4() // fallback: random

```

G. Job Execution Layer

A *WorkerThread* daemon runs on each node and polls its local `data_store` for jobs with status `PENDING`. Claims are atomic: the worker transitions a job to `CLAIMED` under a reentrant lock before dispatching it to a bounded thread pool (default 4 workers). The lifecycle is: `PENDING` → `CLAIMED` → `RUNNING` → `{DONE, FAILED}`.

Failed jobs are retried up to $R_j = 3$ times with a 2-second delay between attempts. Jobs that remain in `RUNNING` for more than 120 seconds are either re-queued (if retry budget remains) or permanently failed. Three built-in job types are supported: `echo` (returns the input message), `sleep` (suspends for a configurable duration), and `compute` (computes $\sum_{i=0}^n i$ as a CPU-bound workload).

H. Dual API and Observability

1) *REST API*: A Flask server on each node exposes two groups of endpoints:

- **Chord RPCs** (`/chord/*`): `find_successor`, `get_predecessor`, `notify`, `get_state`, `ping`, `bulk_put`, `update_successor`, `update_predecessor`.
- **Client API**: `POST /jobs`, `GET /jobs/{id}`; `POST /tasks`, `GET|PATCH|DELETE /tasks/{id}`; `GET /api/ring` (BFS topology walk); `DELETE /api/nodes/{addr}` (graceful node removal); `GET /metrics` (JSON snapshot for agents); `GET /prom_metrics` (Prometheus text format).

HTTP transport uses exponential-backoff retries (3 attempts, 0.1 s base delay) at the transport layer, providing robustness against transient network interruptions without requiring changes at the application layer.

2) *gRPC API*: A `TaskService` proto mirrors the REST task CRUD operations. An `InternalReplicationService` handles replica writes between nodes. Payloads are JSON-serialized inside proto string fields, enabling rapid iteration without full schema regeneration.

3) *Observability*: A web dashboard served at `GET /` provides a real-time ring topology view using `/api/ring` BFS data. Prometheus metrics (via `prometheus-client`) expose per-node counters for jobs completed, jobs

failed, and queue depth. A docker-compose stack in observability/ brings up Prometheus and Grafana for production monitoring. Every agent decision is appended to a structured JSONL log (agent_decisions.jsonl) with timestamp, agent name, strategy used, decision output, and latency, enabling offline audit and replay.

ACKNOWLEDGMENT

The preferred spelling of the word “acknowledgment” in America is without an “e” after the “g”. Avoid the stilted expression “one of us (R. B. G.) thanks ...”. Instead, try “R. B. G. thanks...”. Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

- [1] V. K. Vavilapalli et al., “Apache Hadoop YARN: Yet Another Resource Negotiator,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2013, pp. 1–16. doi: 10.1145/2523616.2523633.
- [2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016. doi: 10.1145/2898442.2898444.
- [3] B. Hindman et al., “Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center,” in *Proc. USENIX NSDI*, 2011, pp. 295–308.
- [4] M. Hashimoto, “Nomad: A Distributed, Highly Available, Datacenter-Aware Scheduler,” HashiCorp Technical Report, 2015. [Online]. Available: <https://www.nomadproject.io>
- [5] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-Free Coordination for Internet-Scale Systems,” in *Proc. USENIX ATC*, 2010, pp. 145–158.
- [6] X. Fong et al., “etcd: A Distributed Reliable Key-Value Store,” GitHub, 2013. [Online]. Available: <https://github.com/etcd-io/etcd>
- [7] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications,” in *Proc. ACM SIGCOMM*, 2001, pp. 149–160. doi: 10.1145/383059.383071.
- [8] P. Maymounkov and D. Mazières, “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric,” in *Proc. Int. Workshop Peer-to-Peer Systems (IPTPS)*, 2002, pp. 53–65. doi: 10.1007/3-540-45748-8_5.
- [9] A. Rowstron and P. Druschel, “Pastry: Scalable, Decentralized Object Location and Routing for Large-Scale Peer-to-Peer Systems,” in *Proc. IFIP/ACM Middleware*, 2001, pp. 329–350. doi: 10.1007/3-540-45518-3_18.
- [10] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, “A Scalable Content-Addressable Network,” in *Proc. ACM SIGCOMM*, 2001, pp. 161–172. doi: 10.1145/383059.383072.
- [11] B. Y. Zhao et al., “Tapestry: A Resilient Global-Scale Overlay for Service Deployment,” *IEEE J. Sel. Areas Commun.*, vol. 22, no. 1, pp. 41–53, 2004. doi: 10.1109/JSAC.2003.818784.
- [12] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-Value Store,” in *Proc. ACM SOSP*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281.
- [13] A. Lakshman and P. Malik, “Cassandra: A Decentralized Structured Storage System,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010. doi: 10.1145/1773912.1773922.
- [14] F. Dabek, M. F. Kaashoek, D. Karger, R. Morris, and I. Stoica, “Wide-Area Cooperative Storage with CFS,” in *Proc. ACM SOSP*, 2001, pp. 202–215. doi: 10.1145/502034.502054.
- [15] A. Rowstron and P. Druschel, “Storage Management and Caching in PAST, a Large-Scale, Persistent Peer-to-Peer Storage Utility,” in *Proc. ACM SOSP*, 2001, pp. 188–201. doi: 10.1145/502034.502053.
- [16] I. Gupta, K. Birman, P. Linga, A. Demers, and R. van Renesse, “Kelips: Building an Efficient and Stable P2P DHT Through Increased Memory and Background Overhead,” in *Proc. Int. Workshop Peer-to-Peer Systems (IPTPS)*, 2003, pp. 160–169. doi: 10.1007/978-3-540-45172-3_15.
- [17] R. Mahmud, K. Ramamohanarao, and R. Buyya, “FogBus: A Blockchain-Based Lightweight Framework for Edge and Fog Computing,” *J. Syst. Softw.*, vol. 154, pp. 22–36, 2019. doi: 10.1016/j.jss.2019.04.050.
- [18] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, “iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments,” *Softw. Pract. Exper.*, vol. 47, no. 9, pp. 1275–1296, 2017. doi: 10.1002/spe.2509.
- [19] N. Wang, B. Varghese, M. Matthaiou, and D. S. Nikolopoulos, “ENORM: A Framework for Edge Node Resource Management,” *IEEE Trans. Serv. Comput.*, vol. 13, no. 6, pp. 1086–1099, 2017. doi: 10.1109/TSC.2017.2753775.
- [20] B. Confais, A. Lebre, and B. Parrein, “An Object Store Service for a Fog/Edge Computing Infrastructure Based on IPFS and a Modified Kademlia DHT,” in *Proc. IEEE IC2E*, 2017, pp. 170–176. doi: 10.1109/IC2E.2017.29.
- [21] M. Villari, M. Fazio, S. Dustdar, O. Rana, and R. Ranjan, “Osmotic Computing: A New Paradigm for Edge/Cloud Integration,” *IEEE Cloud Comput.*, vol. 3, no. 6, pp. 76–83, 2016. doi: 10.1109/MCC.2016.124.